

BT / BST



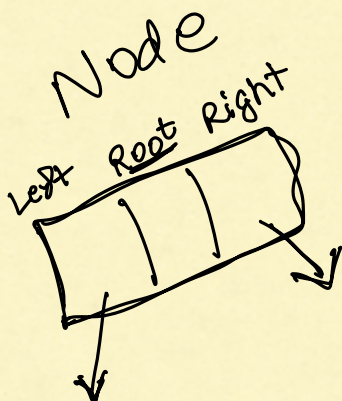
li) Pre order

** lii) In order

liii) Post order

Left Root Right

Trick → put everything
inside an
array/set
↓
modification

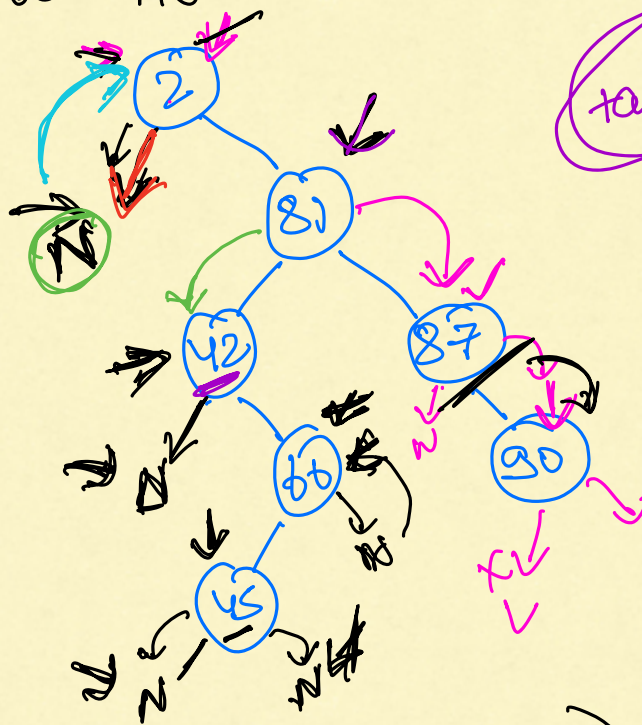


Inorder ⇒ BST always a sorted order

Search a node in BST

N | N | N

✓ Left ✓
 ✓ Root ✓
 ✓ Right ✓
 n5



tar = 87

root
 [2]
 Left root.left
 Right root.right

[2, 42,
 45, 66, 81,
 87, 90]

```
def inorder(root, ans):
    if root == None:
        return
```

inorder(root.left, ans)

ans.append(root.data)

inorder(root.right, ans)

return

if root.data == tar
 return True

s.add(root.data)

ans = [2, 42, 45, 66, 81, 87, 90]

s = set()
 ans = []

sorted

inorder(root, ans)

if i in ans

(log n)

inorder
 $O(N)$
(sorted)

searching
 $O(N)$
(searching)

Binary
search

set/dict

$S = \{2, 4, 7, 15, 66, 81, 87, 90\}$

if i in $S \Rightarrow$ set
 $\hookrightarrow O(1)$

if i in arr
 $\hookrightarrow O(N)$

$d = \{2: \text{True}, 42: \text{True}, 15: \text{True}\}$

$\{60: \text{True}, 81: \text{True}, 87: \text{True}, 90: \text{True}\}$

return
 $d[\text{tax}]$
key -1

if i in d
 $\hookrightarrow O(1)$

$O(N) + O(N) = O(2N)$
 $O(N) + O(1) = O(N)$
 $O(N) \approx O(2N)$

k^{th} largest element in BST



$k=2$
 $\hookrightarrow \underline{\underline{4}}$

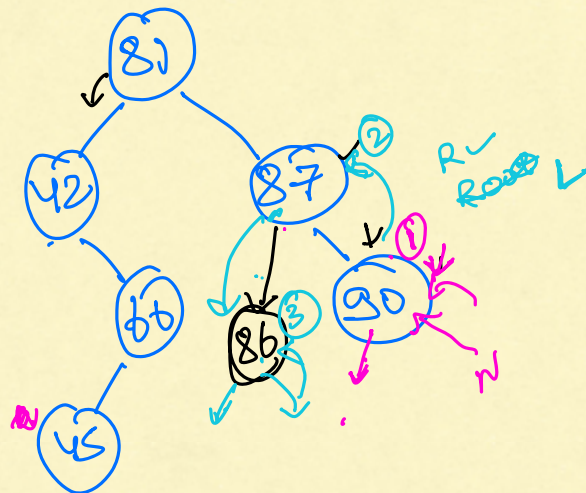
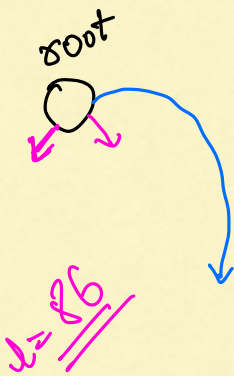
inorder $\Rightarrow [2, 4, 9]$
 3rd 2nd 1st
 $\text{return } \text{arr}[k-1]$

$k=1$

$\text{arr}[k-1]$
 0

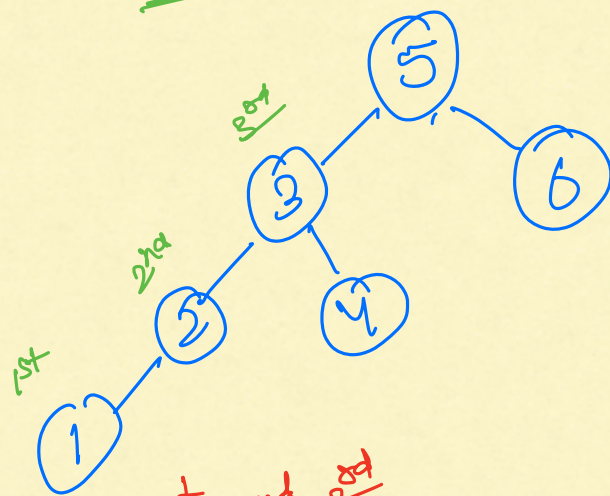
1st $\text{arr}[k]$
 $\text{arr}[1]$

$[10, 20, 30, 40, 50, 60]$



$[42, 45, 66, 81, 87, 90]$
 3rd 2nd 1st

K^{th} small smallest element

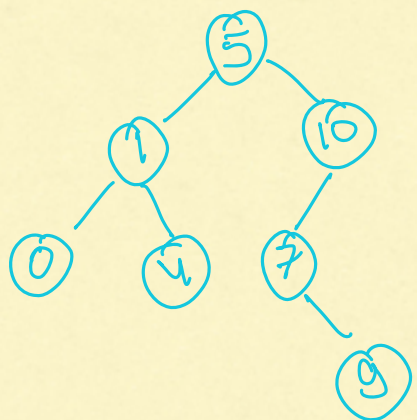


$K=3$
 $\rightarrow \text{ans} = \underline{\underline{3}}$

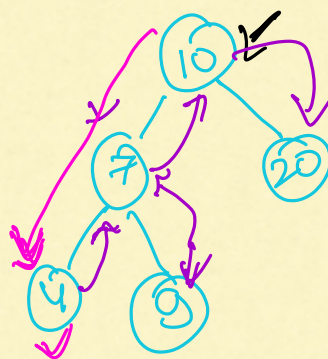
return
ans[K-1]

inorder = [1, 2, 3, 4, 5, 6]
idx ↑ 1 2 3 4 5
 Small

Find Common Nodes in BST



BST 1



BST 2

that
are
common
in
both
BST

if 10 in set
→ 04)

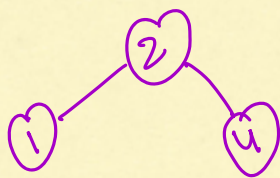
in order = [0, 1, 4, 5, 7, 9, 10]

if i is in set
→ T/F

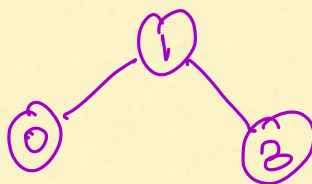
if 4 is in set
→ 04)

~~[10, 9]~~
✓ [4, 7, 9, 10]

All Element in Two BST



BST 1



BST 2

all Integer from both
BST1 & BST2
array, containing all
integers in sorted order

[0, 1, 1, 2, 3, 4]

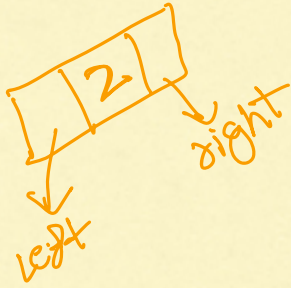
[0, 1, 1, 2, 3, 4]
[0, 1, 2, 4]

i <= j
ans.append(i)
i=i+1
else
ans.append(j)
j=j+1

merge
 $L1 + L2 = L1 \cup L2 \text{ sorted}$

TC
↓
L1: [1, 2, 4] in order
L2: [0, 1, 3] in order
↓
ans = []

Insert a node



class Node
List Node
Node

def func(root, key)

if root.data == key:

return

if root.left == None and root.right == None

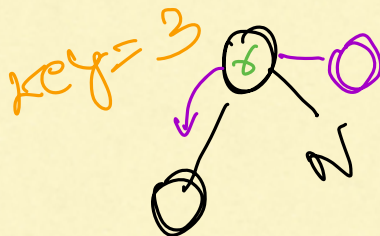
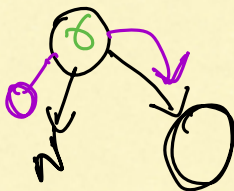
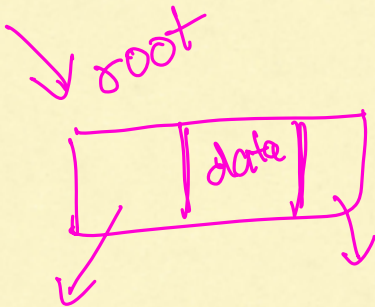
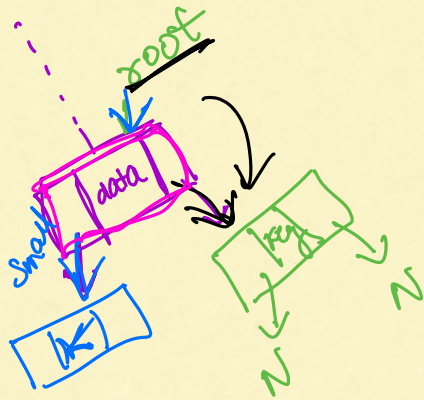
if key > root.data:

root.right = Node(key)

else:

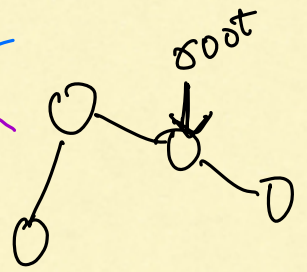
root.left = Node(key)

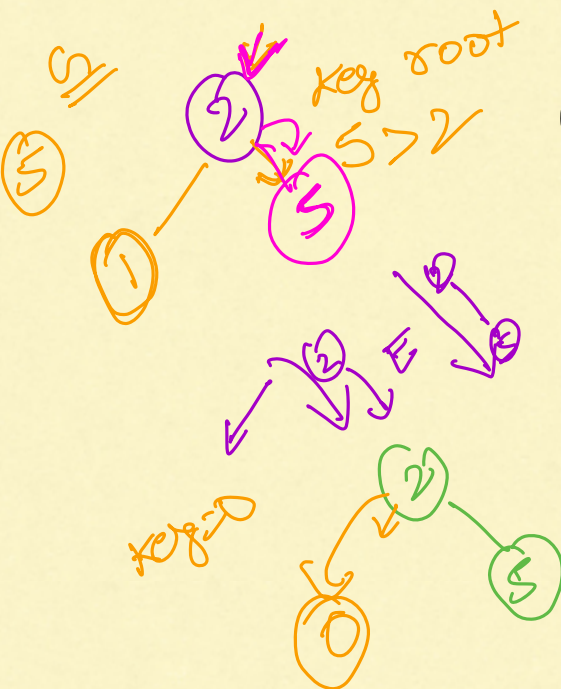
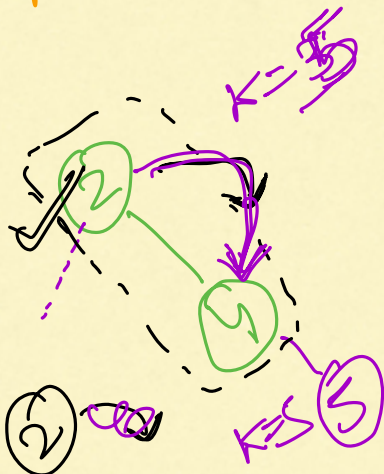
return



root

key



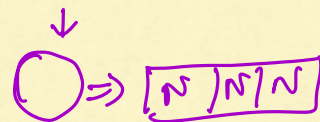


if $key > root.data$ and $root.right \neq None$
 (right) $return fun(root.right, key)$
 return

if $key > root.data$ and $root.right == None$
 $root.right = Node(key)$
 return

if $key < root.data$ and $root.left \neq None$
 (left) $return fun(root.left, key)$
 return

if $key < root.data$ and $root.left == None$
 $root.left = Node(key)$
 return

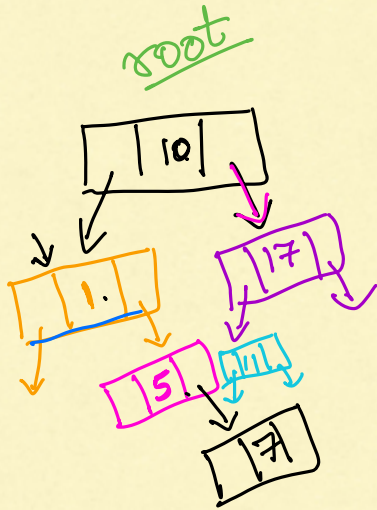


if $root == None$
 $root = Node(0)$



else :

[10, 1, 17, 5, 11, 7]



$i < \text{root}$ and $\text{root.left} == \text{None}$
 $\text{root.left} = \text{Node}(i)$
 $i < \text{root}$ and $\text{root.left} != \text{None}$:
 $\text{fun}(\text{root.left}, i)$

$i > \text{root}$ and $\text{root.right} == \text{None}$
 $\text{root.right} = \text{Node}(i)$

$i > \text{root}$ and $\text{root.right} != \text{None}$
 $\text{fun}(\text{root.right}, i)$